

PROJECT REPORT

Agile Banking System

A PROJECT REPORT ON "TRUST-BANK" AUTOMATED BANKING SYSTEM *(Developed using Agile Scrum Methodology)*

Submitted By: Neeraj Sahu

1. Abstract

The banking sector demands software that is not only functional but also impeccably secure and compliant with financial regulations. This project, "**Trust-Bank**," is a web-based Automated Banking System designed to allow customers to manage accounts, transfer funds, and view statements online.

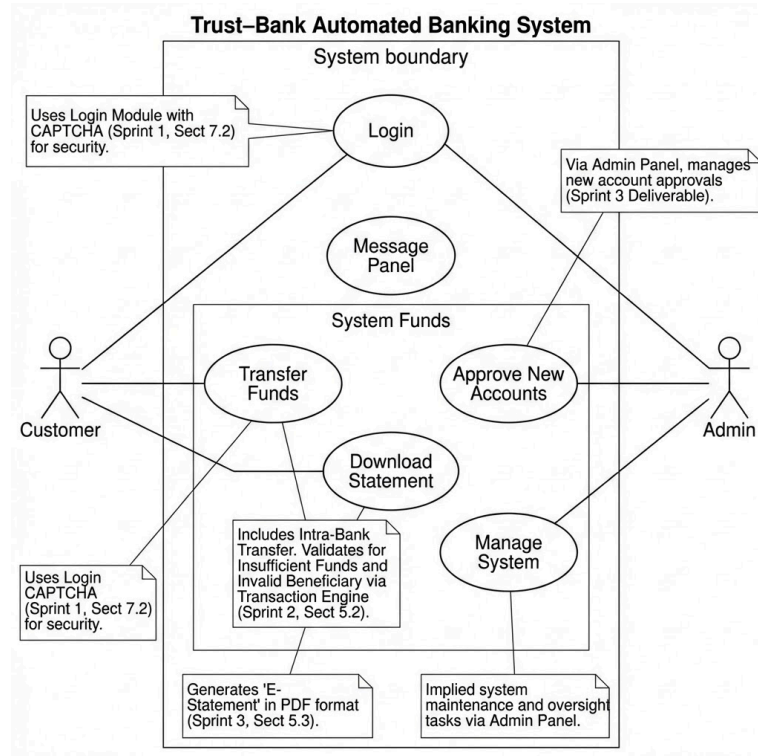
Given the critical nature of financial data, the project was executed using the **Agile Scrum Methodology**. This iterative approach allowed the team to prioritize security features (like Two-Factor Authentication) in the very first sprint. By testing small increments of code frequently, we ensured that critical vulnerabilities were identified and patched early in the lifecycle, reducing the risk of catastrophic failure in production.

Keywords: Agile, Scrum, FinTech, Security, Fund Transfer, Encryption.

2. Banking System Overview

2.1 Introduction

"Trust-Bank" is a simulation of a Core Banking Solution (CBS). It bridges the gap between the physical bank branch and the customer. It allows users to perform day-to-day banking activities—checking balances, transferring money, and downloading reports—without visiting a physical location.



2.2 Problem Statement

Developing banking software via the **Waterfall Model** is dangerous.

1. **Security Risks:** In Waterfall, security testing often happens at the end. If a fundamental flaw is found in the architecture, the entire system must be rebuilt.
2. **Regulatory Changes:** RBI or government guidelines change frequently. Waterfall cannot adapt to new compliance rules mid-project.
3. **User Trust:** A clumsy UI can make users distrust the bank. Agile allows for continuous UI refinement based on feedback.

2.3 The Agile Solution

We utilized **Agile Scrum** to treat "Security" and "Compliance" as continuous activities, not just final checkboxes. Every sprint included a "Security Review" task to ensure data integrity.

3. Agile Principles Applied

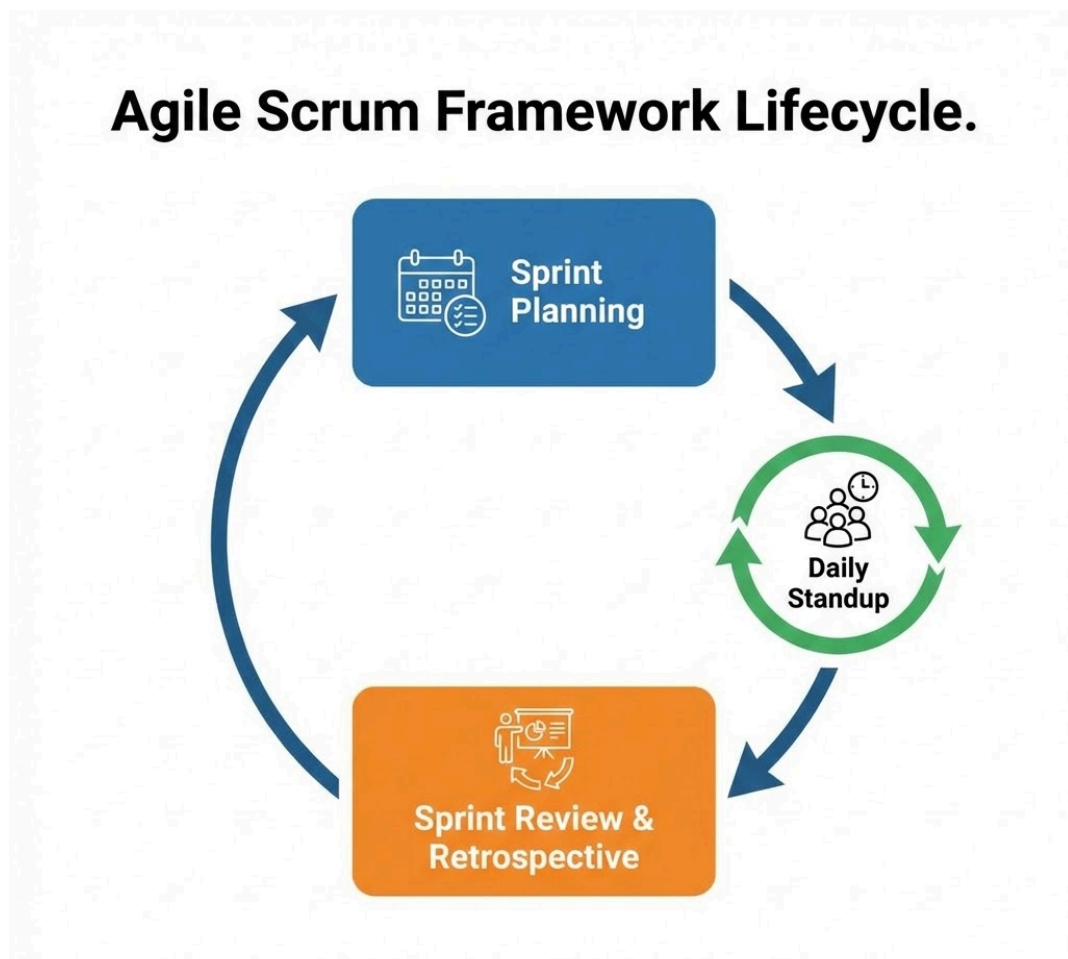
3.1 The Scrum Lifecycle

The project was divided into 3 Sprints of 2 weeks each.

- **Product Owner:** Defined the banking rules (e.g., "Minimum balance must be ₹1000").
- **Scrum Master:** Ensured the team followed secure coding practices (e.g., "No SQL Injection").
- **Dev Team:** Built the full-stack application.

3.2 Key Agile Ceremonies

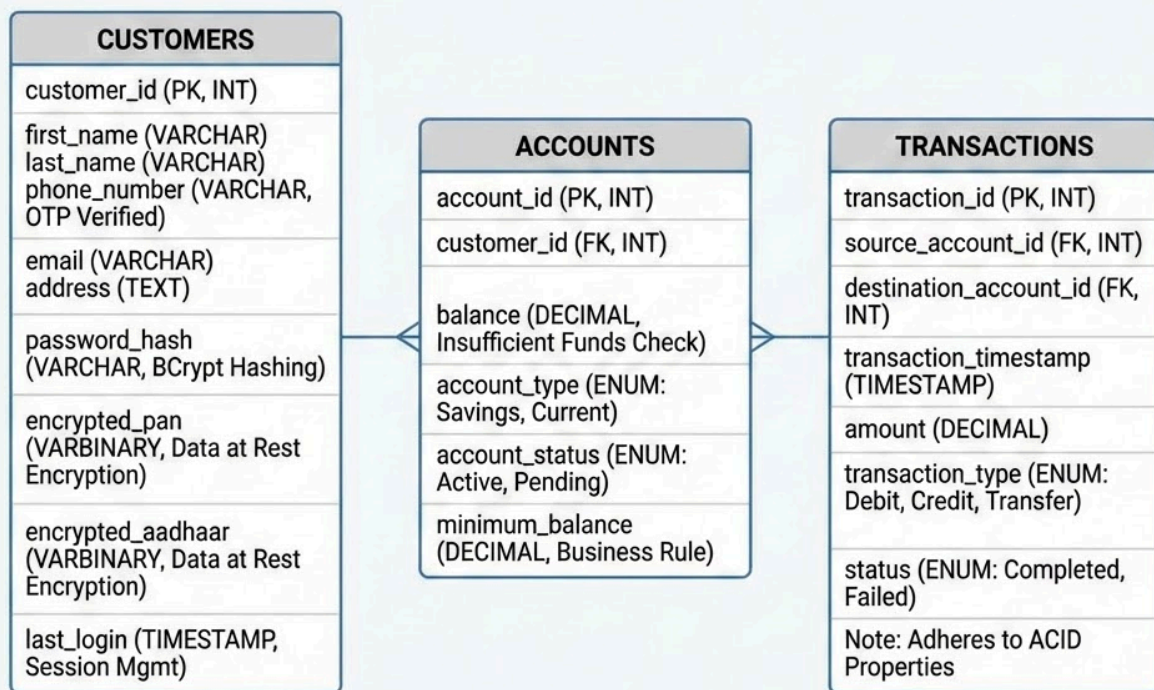
1. **Sprint Planning:** Prioritizing "High Risk" items first. We decided to build the **Transaction Engine** before the **UI Dashboard** to ensure the math was correct.
2. **Daily Standup:** Used to flag "Blockers" immediately.
3. **Sprint Retrospective:** After Sprint 1, we realized our session timeout was too long (30 mins). We reduced it to 5 mins in the next sprint for better security.



4. Sprint Planning and Delivery

Sprint 1: Security & Foundation (Weeks 1-2)

- **Goal:** Secure Authentication & Account Creation.
- **Deliverables:**
 - User Registration with Password Hashing (BCrypt).
 - Login Module with CAPTCHA.
 - Database Design for **Accounts** and **Users**.
- **Testing:** Verified that passwords are **never** stored in plain text.



Sprint 2: The Transaction Engine (Weeks 3-4)

- **Goal:** Fund Transfers (The Core).
- **Deliverables:**
 - "Add Beneficiary" feature.
 - Internal Fund Transfer logic (Debit Sender -> Credit Receiver).
 - ACID Properties (Atomicity, Consistency, Isolation, Durability) implementation in SQL.
- **Challenge:** Handling concurrent transactions (Race Conditions).
- **Fix:** Implemented Database Row Locking.

Sprint 3: Reporting & Statements (Weeks 5-6)

- **Goal:** User Experience & Admin.
- **Deliverables:**
 - PDF Statement Generation.
 - "Transaction History" Search/Filter.
 - Admin Panel to approve new accounts.
- **Outcome:** System ready for UAT (User Acceptance Testing).

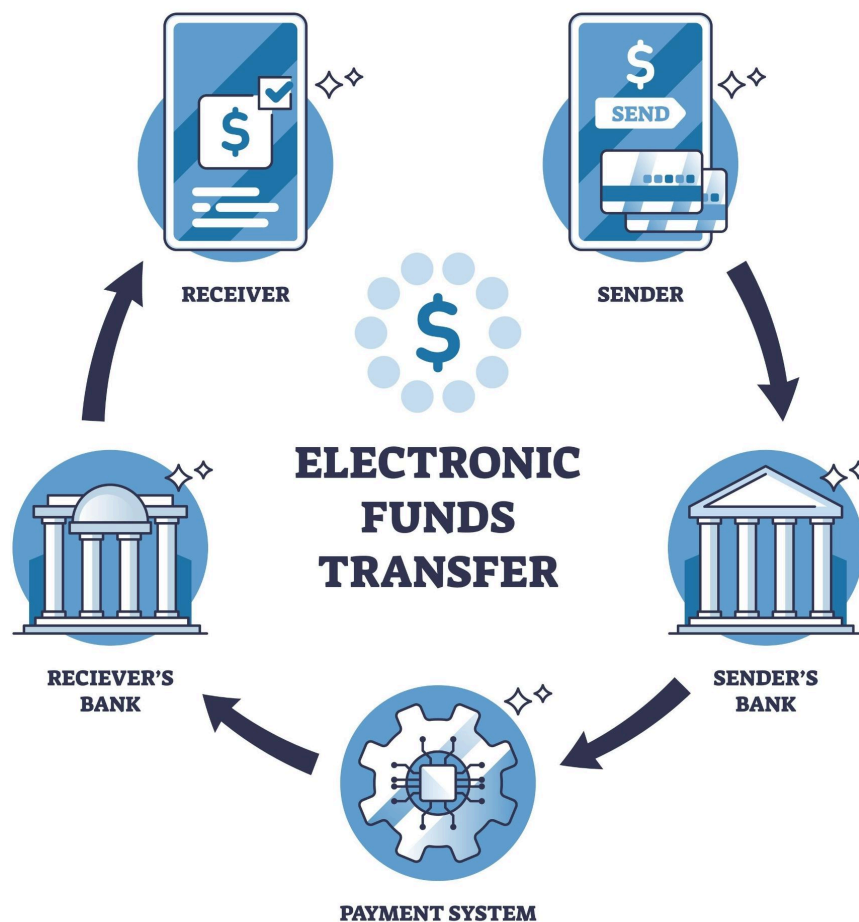
5. Core Banking Features

5.1 Account Management

- Users can view real-time balances for Savings and Current accounts.
- **Profile Management:** Update address and phone number (requires OTP verification).

5.2 Fund Transfer System

- **Intra-Bank Transfer:** Instant money transfer between accounts within "Trust-Bank."
- **Validation:** System checks for "Insufficient Funds" and "Invalid Beneficiary" before processing.



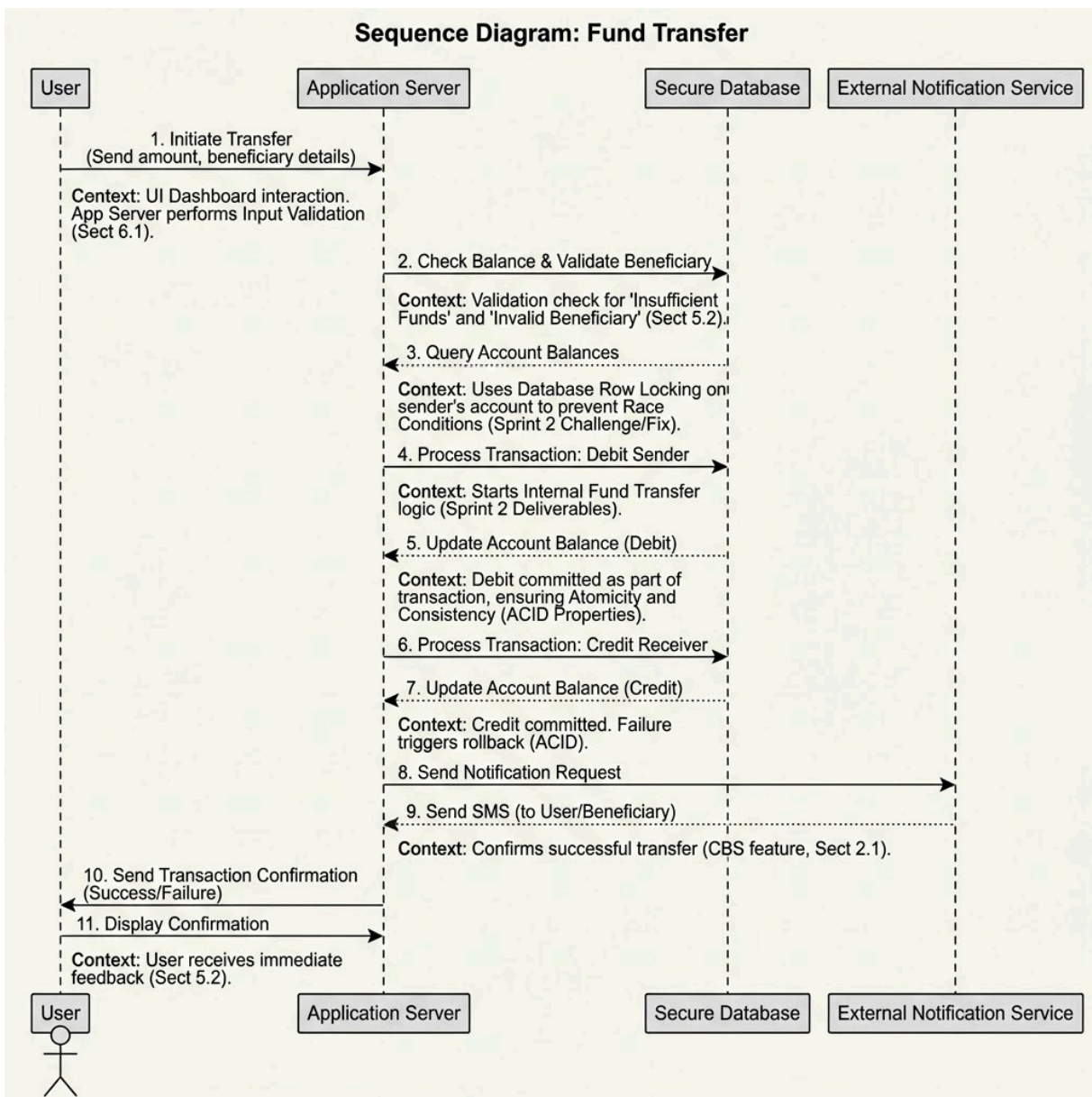
5.3 Transaction History

- Users can download an "E-Statement" in PDF format.
- Filters available for "Last 10 Transactions" or "Date Range."

6. Security Considerations (Crucial Section)

Banking software lives and dies by security. We implemented:

1. **Input Validation:** To prevent **SQL Injection** and **XSS Attacks**. All inputs are sanitized.
2. **Session Management:**
 - Auto-logout after 5 minutes of inactivity.
 - Disabling the "Back Button" after logout to prevent viewing cached pages.
3. **Encryption:**
 - **Data in Transit:** Uses HTTPS (SSL/TLS).
 - **Data at Rest:** Sensitive info (PAN, Aadhaar) is encrypted in the database.



7. Benefits of Agile Development in Banking

7.1 Managing Compliance

If the government announces a new tax rule (e.g., "TDS on transfer > ₹50k"), Agile allows us to add this logic in the *very next sprint*. Waterfall would delay compliance by months.

7.2 Risk Mitigation

By releasing the "Login Module" in Sprint 1, we could test it against brute-force attacks immediately. We didn't wait until the end to find out our login was weak.

7.3 Customer-Centric Design

We showed the "Transfer Screen" to a focus group in Sprint 2. They found it confusing. We simplified the UI in Sprint 3. This responsiveness creates a better product.

8. Conclusion

The "Trust-Bank" project demonstrates that **Agile is essential for FinTech development**. It allows for the rigorous testing of security features and the flexibility to adapt to financial regulations. By prioritizing "Secure Code" in every sprint, we delivered a banking platform that is not just functional, but trustworthy.